

Acceleration of Quantum Kernel Methods for Image Classification Tasks

Project Work Parallel Programming

Fouepe Dylan

 University of Florence

 Parallel Programming – 2025

Overview

1. Context & Scope
2. Experimental Methodology
3. HPC Architecture
4. Performance Benchmark
5. Classification Results
6. Failure Analysis: Kernel Collapse
7. Conclusion & Perspectives

Context & Scope

💡 Quantum Kernel SVMs

- Project classical data into a 2^Q -dimensional Hilbert space.
- Fidelity kernel: $k(x_i, x_j) = |\langle \psi(x_i) | \psi(x_j) \rangle|^2$.
- At $Q = 16$: 65,536 complex amplitudes per state.
- Guaranteed convex SVM optimization.

⚠️ HPC Bottleneck

- Gram matrix cost: $\mathcal{O}(N^2 \cdot 2^Q)$.
- GPU/CUDA acceleration is essential at scale.

🎯 Three Research Questions

1. How do classical and quantum-kernel SVMs compare on the same binary tasks?
2. Which backend offers the best throughput-runtime trade-off at 16 qubits?
3. Which empirical signals indicate kernel concentration?

Experimental Methodology

Three datasets, binarized into three difficulty levels:

Dataset	Easy	Med	Hard
Fashion-MNIST	0 vs 1	3 vs 8	0 vs 6
CIFAR-10	1 vs 6	0 vs 2	3 vs 5
SVHN	1 vs 0	2 vs 7	3 vs 5

✓ Evaluation Protocol

- 3-fold Cross-Validation
- Optuna for hyperparameter C
- **Key Metrics:** F1-score, AUC, runtime, SV fraction, kernel std

Quantum Configuration

- $Q = 16$ qubits $\rightarrow 2^{16} = 65,536$ complex amplitudes
- **YZ ansatz** with entangling layers
- Fixed weights at initialization (Kernel-based approach)

HPC Environment

- **GPU:** NVIDIA RTX 6000 Ada Generation (48 GB VRAM)
- **CPU:** AMD EPYC 7343 16-Core (64 threads)
- **Software Stack:** CUDA 13.1

Preprocessing Pipeline & Fidelity Kernel

Feature pipeline:

1. Extract pixels and flatten
2. PCA to $d = 16$ components
3. Range scaling
4. Encode via *angle map*, *Y map* or *YZ map*
5. Simulate states via PennyLane

Fidelity kernel:

$$K_{ij} = \left| \sum_{k=1}^{2^Q} S_A[i, k] \overline{S_B[j, k]} \right|^2$$

Training protocol:

- Train/test split from dataset
- 3-fold CV on training partition
- Hyperparameter search (C) via Optuna
- Final re-fit on train+val, eval on test set

i Kernel-based, not variational

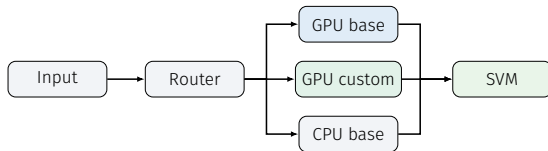
Weights are fixed at init – kernel method, not a training loop.

HPC Architecture

Unified Backend API

Three execution paths, same normalization and SVM:

- **CPU base** – NumPy reference, no VRAM tiling
- **GPU base** – PyTorch GPU matmul + pinned memory + CUDA streams
- **GPU custom** – PyTorch states → DLPack → CuPy → custom CUDA kernel (FP64)



SVM solver always runs on CPU.

⚙️ Backend Tuning

Auto-tiling

VRAM budget limit

Persistent Cache

(T_m, T_n) kernel sizes

Bulk Precompute

Minimize PyTorch overhead

Stream Pool

Async overlap

Optimal: `vram_fraction=0.95, num_streams=1`

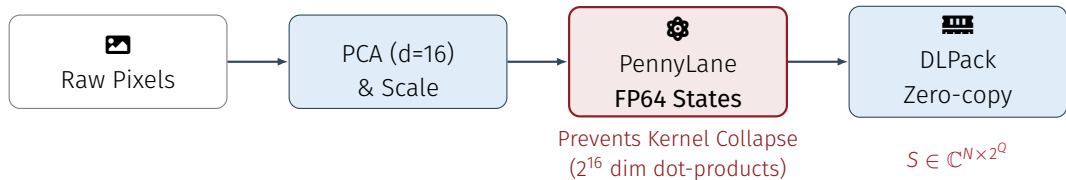
⚖️ Shared Normalization

All backends use identical post-processing:

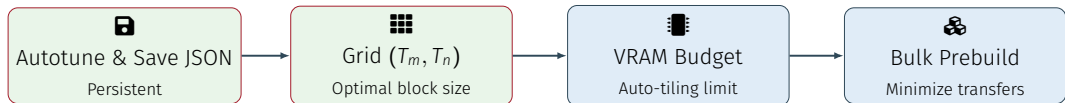
$$\hat{K}_{ij} = \frac{K(X, Y)_{ij}}{\sqrt{K(X, X)_{ii} K(Y, Y)_{jj}}}$$

- NaN/Inf checks before writing
- Diagonal validation before SVM fit
- Guarantees fair comparison

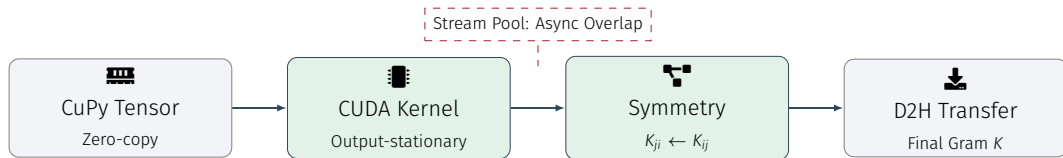
GPU Custom: Phase 1 – Host Preprocessing



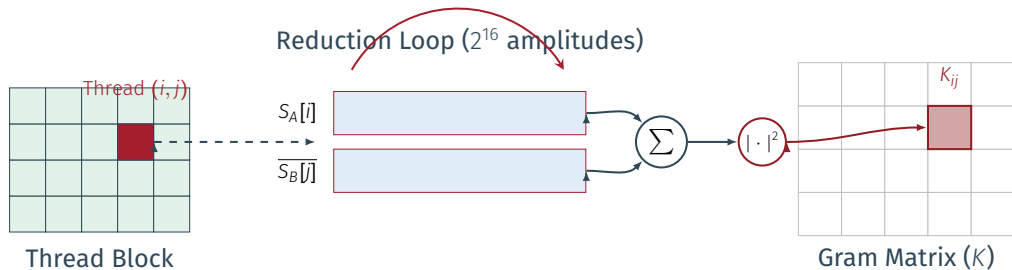
GPU Custom: Phase 2 – Orchestration



GPU Custom: Phase 3 – Device Core



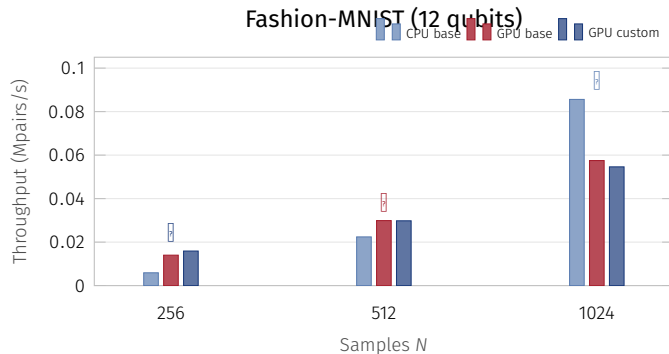
GPU Custom: Visual Thread Mapping



- **Output-Stationary:** 1 thread $\rightarrow$ 1 cell (K_{ij}).
- **Zero Collision:** Direct VRAM write (no atomic operation).

Performance Benchmark

Throughput by Dataset & Backend (Mpairs/s)



CIFAR-10 & SVHN (16 qubits):

N	GPU base	GPU cust.
CIFAR-10		
512	0.0175	0.0112
SVHN		
512	0.0175	0.0124

= best at

each size

Pattern:

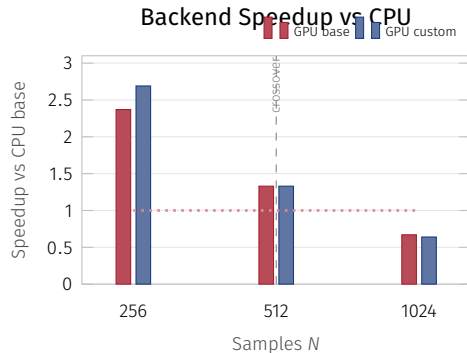
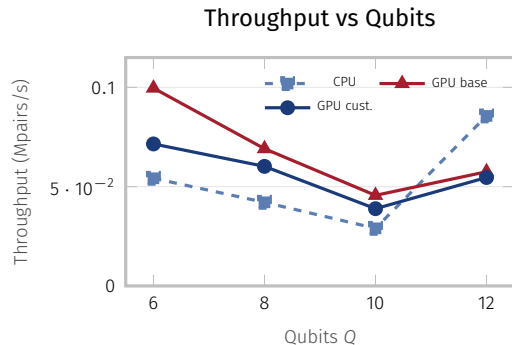
$N=256 \rightarrow$ GPU custom

$N \geq 512 \rightarrow$ GPU base

(except Fashion $N=1024$)

$\rightarrow$ CPU cache wins)

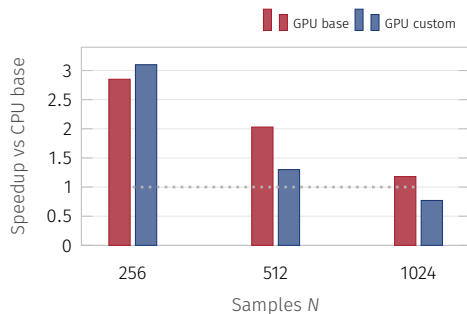
Benchmark Profile – Fashion-MNIST (12 qubits)



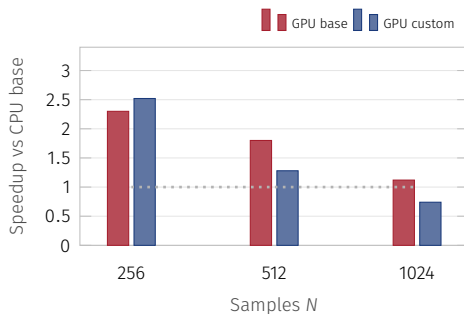
GPU custom leads at **N=256** (2.69 $\times$). GPU base at **N=512**. CPU cache recovers at **N=1024**.

Benchmark Profiles – CIFAR-10 & SVHN (16 qubits)

CIFAR-10

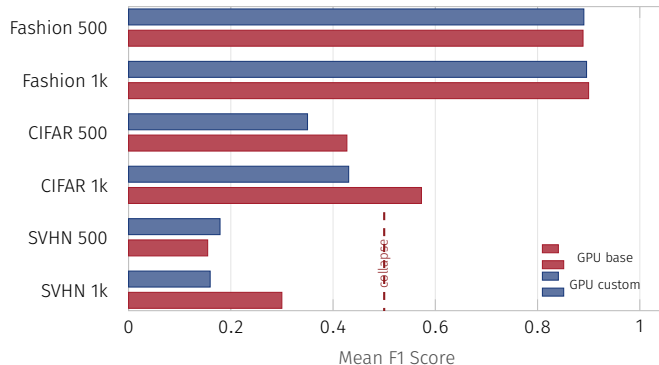


SVHN



GPU base dominates at $N \geq 512$. GPU custom overhead (DLPack + CuPy) outweighs kernel gains at large N .

Quantum Backend Quality – F1 by Dataset & Size



Fashion [?]

Both backends strong

F1 up to **0.9653**

AUC up to **0.9956**

CIFAR-10 ~

GPU base ahead

F1 $\in [0.35, 0.57]$

Gap at 1000 samples

SVHN [?]

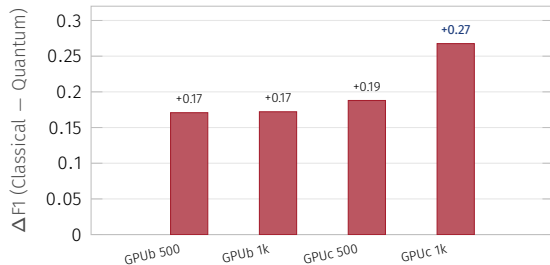
Kernel collapse

F1 ≤ 0.30

AUC ≈ 0.48

Classical vs Quantum: Paired Comparison

Classical advantage grows with N



Classical wins 8/9

Both sizes. Gap **widens** at $N=1000$ for GPU custom (+0.27).

Runtime (classical faster):

	N=500	N=1000
vs GPU base	-188s	-275s

Key:

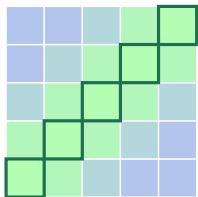
Quantum cost is representational, not HPC.

Acceleration $\neq$ expressivity.

Hard Tasks: Kernel Collapse – Quantitative Signals

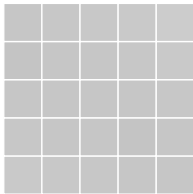
CIFAR-10 Hard, 6 layers — collapse confirmed:

Healthy (Fashion)



std ≈ 0.12

Collapsed (CIFAR hard)



std $\approx 10^{-4}$

Collapse signals

kernel std	= 0.0005
SV fraction	= 1.0000
test F1	= 0.0000
test AUC	= 0.4757

Contributing factors:

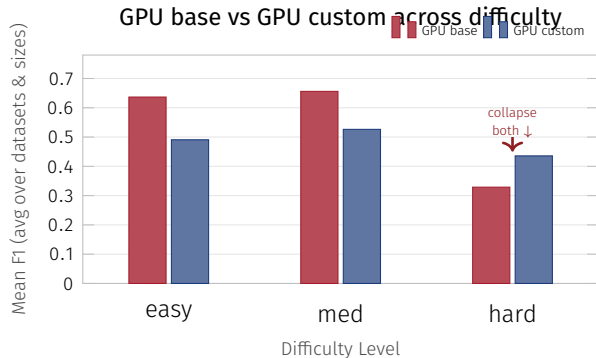
- PCA/grayscale info loss
- Generic YZ ansatz (not image-aware)
- Deeper circuits $\rightarrow$ concentration
- Global fidelity kernel limitation

States become indistinguishable in $\mathcal{H}$

$\Rightarrow$ SVM sees no class signal.

Acceleration cannot fix this.

Difficulty-Level Summary – F1 Across Backends



Easy & Med

GPU base faster **and** more accurate.
 $F1 \in [0.49, 0.66]$.

Hard

Both backends collapse. F1 drops,
AUC $\rightarrow 0.5$. Representational
bottleneck, not HPC.

	F1	AUC
GPU base easy	0.637	0.650
GPU cust. easy	0.491	0.633

Conclusion & Perspectives

Conclusion

- ✔ **HPC Pipeline:** Three backends share a unified API. GPU paths dominate small sizes; ranking varies with dataset and sample count.
- 📈 **Fashion quality:** Both quantum backends competitive (F1 up to **0.9653**, AUC up to **0.9956** for GPU base, size 1000).
- ❗ **Classical dominates:** RBF SVM wins 8/9 paired comparisons at both sample sizes. Gap widens at 1000 samples.
- ⊘ **Kernel collapse on hard tasks:** Concentration signals ($\text{std} \sim 10^{-4}$, SV frac = 1). Acceleration alone does not fix this.

→ Better Feature Maps

- Projected/local kernels to reduce concentration
- Geometry-aware ansatz for image structure
- Stronger classical front-end before quantum embedding

→ Richer Diagnostics

- Gap trajectories and kernel spectra
- SV dynamics to detect collapse earlier
- Broader benchmark grids per dataset

→ HPC Scaling

- Multi-GPU / MPI for large Gram builds
- GPU-side SVM solvers to remove CPU bottleneck

Thank You!

 github.com/DylanUnifi

 University of Florence

Questions?